

Notebook: all_algo_comparsons/decision_tree_comparison.ipynb

[--- MARKDOWN CELL ---]

Decision Tree - Algorithm Comparison

Comparing Decision Tree performance on Breast Cancer and Wine datasets using sklearn.

[--- CODE CELL ---]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer, load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
np.random.seed(42)
```

[--- MARKDOWN CELL ---]

Dataset 1: Breast Cancer

[--- CODE CELL ---]

```
bc = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(bc.data, bc.target, test_size=0.2, random_state=42)
# Train with different max_depth values
depths = [None, 3, 5, 10]
bc_results = []
for depth in depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)
    acc = dt.score(X_test, y_test)
    cv_scores = cross_val_score(dt, bc.data, bc.target, cv=5)
    bc_results.append({'max_depth': str(depth), 'test_acc': acc, 'cv_mean': cv_scores.mean()})
bc_df = pd.DataFrame(bc_results)
print("Breast Cancer Results:")
print(bc_df.to_string(index=False))
```

[--- MARKDOWN CELL ---]

Dataset 2: Wine

[--- CODE CELL ---]

```
wine = load_wine()
X_train, X_test, y_train, y_test = train_test_split(wine.data, wine.target, test_size=0.2, random_state=42)
wine_results = []
for depth in depths:
    dt = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt.fit(X_train, y_train)
    acc = dt.score(X_test, y_test)
    cv_scores = cross_val_score(dt, wine.data, wine.target, cv=5)
    wine_results.append({'max_depth': str(depth), 'test_acc': acc, 'cv_mean': cv_scores.mean()})
wine_df = pd.DataFrame(wine_results)
print("Wine Results:")
print(wine_df.to_string(index=False))
```

[--- CODE CELL ---]

Comparison plot

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Breast Cancer
x = np.arange(len(depths))
axes[0].bar(x - 0.2, bc_df['test_acc'], 0.4, label='Test Accuracy', color='steelblue')
axes[0].bar(x + 0.2, bc_df['cv_mean'], 0.4, label='CV Mean', color='coral')
axes[0].set_xticks(x)
axes[0].set_xticklabels(bc_df['max_depth'])
axes[0].set_xlabel('max_depth')
axes[0].set_ylabel('Accuracy')
axes[0].set_title('Breast Cancer - Decision Tree')
axes[0].legend()
axes[0].set_ylim(0.8, 1.05)
axes[0].grid(True, alpha=0.3, axis='y')

# Wine
axes[1].bar(x - 0.2, wine_df['test_acc'], 0.4, label='Test Accuracy', color='steelblue')
axes[1].bar(x + 0.2, wine_df['cv_mean'], 0.4, label='CV Mean', color='coral')
axes[1].set_xticks(x)
axes[1].set_xticklabels(wine_df['max_depth'])
axes[1].set_xlabel('max_depth')
axes[1].set_ylabel('Accuracy')
axes[1].set_title('Wine - Decision Tree')
axes[1].legend()
axes[1].set_ylim(0.7, 1.05)
axes[1].grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Visualize a decision tree
dt_bc = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_bc.fit(bc.data, bc.target)

plt.figure(figsize=(20, 10))
plot_tree(dt_bc, feature_names=bc.feature_names, class_names=bc.target_names,
          filled=True, rounded=True, fontsize=10)
plt.title('Decision Tree - Breast Cancer (max_depth=3)')
plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Feature importance
dt_full = DecisionTreeClassifier(random_state=42)
dt_full.fit(bc.data, bc.target)

importances = dt_full.feature_importances_
sorted_idx = np.argsort(importances)[::-1][:10]

plt.figure(figsize=(12, 6))
plt.barh(range(10), importances[sorted_idx], color='steelblue')
plt.yticks(range(10), [bc.feature_names[i] for i in sorted_idx])
plt.xlabel('Feature Importance')
plt.title('Top 10 Feature Importance - Decision Tree (Breast Cancer)')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

[--- CODE CELL ---]

```
# Summary
print("\nDecision Tree Comparison Summary:")
print("="*60)
print("\nBreast Cancer - Best max_depth=3:")
print(f"  Test Accuracy: {bc_df[bc_df['max_depth']==3]['test_acc'].values[0]:.4f}")
print(f"  5-Fold CV Mean: {bc_df[bc_df['max_depth']==3]['cv_mean'].values[0]:.4f}")
```

```
print("\nWine - Best max_depth=5:")
print(f"  Test Accuracy: {wine_df[wine_df['max_depth']=='5']['test_acc'].values[0]:.4f}")
print(f"  5-Fold CV Mean: {wine_df[wine_df['max_depth']=='5']['cv_mean'].values[0]:.4f}")
```